ARCHITECTURAL MONOGRAPH // SYSTEMS & INFRASTRUCTURE

Distributed Systems & Storage Internals: Production Engineering Reference

Deep-Dive Mechanics of Storage Engines, MVCC, Consensus, Streaming Logs, Kernel Networking, and Queueing Theory

1. Distributed Storage Engines: B+ Trees vs. LSM Trees

STORAGE_INTERNALS

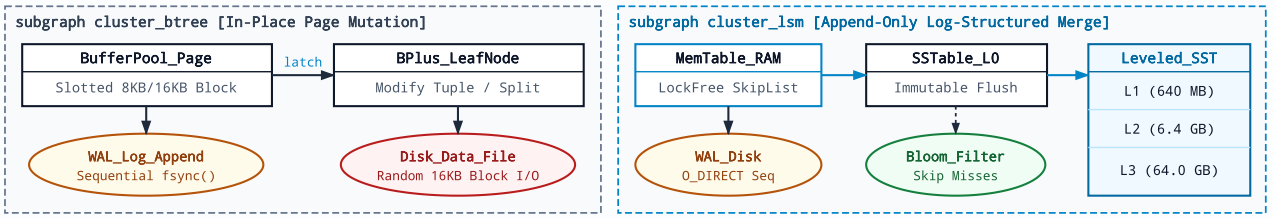


FIG 1.1: Storage Engine Write & Compaction Topologies (B+ Tree vs. RocksDB LSM)

```
layout=dot;
rankdir=LR
```

1.1 B+ Tree Internals (InnoDB, PostgreSQL Heap + B-Tree)

- **Page Layout & Slotted Pages:** Storage engines allocate fixed-size blocks (InnoDB uses 16 KB; PostgreSQL uses 8 KB). Within an 8 KB PostgreSQL page, entries are managed via a *slotted-page architecture*: a line-pointer array at the page header grows downward, while variable-length tuple payloads grow upward from the page trailer, preserving contiguous free space in the middle.
- **The B+ Tree Structural Invariant:** Unlike classical B-trees where keys and data coexist in internal nodes, a B+ Tree stores row payloads or tuple pointers exclusively in leaf nodes. Internal nodes hold only routing keys and child page pointers. This maximizes node **fan-out** (B). With $B \approx 300–500$, tree height rarely exceeds 3 to 4 levels ($h \approx log_B N$), bounding point lookups to 3–4 page accesses.
- **Write Amplification (WA) & Page Splits:** Inserting a key into a full leaf page (100% fill factor) triggers a page split into two 50%-full pages. The engine latches the parent routing node, updates separator keys, allocates new blocks, and logs WAL records. A single 50-byte row mutation flushes an entire 16 KB page during checkpointing, yielding a high Write Amplification Factor ($WAF = Bytes_{storage} / Bytes_{ingested}$).

1.2 LSM Tree Internals (RocksDB, Apache Cassandra, ScyllaDB)

- **MemTable & Immutable Flush:** Writes avoid random in-place disk mutation. Every mutation appends sequentially to an on-disk Write-Ahead Log (WAL) for crash durability while inserting into an in-memory concurrent **Lock-Free SkipList** ($O(log N)$ insert without global locks). Once the MemTable reaches its memory threshold (e.g., 64 MB), it freezes and flushes sequentially to disk as an immutable Level 0 (L_0) Sorted String Table (SSTable).
- **SSTable Internal Anatomy:** An SSTable comprises compressed 4 KB data blocks, a sparse index block mapping keys to block file offsets, and an embedded **Bloom filter block** (configured at ~10 bits/key for a 1% false-positive rate) that rejects non-existent point lookups before touching data blocks.
- **Compaction Mechanics & The RUM Conjecture:**
  - *Size-Tiered Compaction (STCS - Cassandra default):* Merges SSTables of similar size once a threshold count (typically 4) accumulates. Yields low WAF on write bursts, but suffers from severe Space Amplification (requiring up to 50% free disk headroom for merge output) and higher Read Amplification due to overlapping key ranges across tables.
  - *Leveled Compaction (LCS - RocksDB default):* Organizes disk into exponentially growing levels ($L_1 = 640\ MB$, $L_2 = 6.4\ GB$, $L_k = 10 \times L_{k-1}$). For all levels $\geq L_1$, key ranges across SSTables are **strictly non-overlapping**. Compaction merges an L_i SSTable with overlapping L_{i+1} tables via multi-way merge-sort. LCS bounds point-read amplification to 1 file check per level and keeps space overhead below ~10%, at the cost of higher WAF

2. Database Internals: ARIES WAL, MVCC & Isolation Anomalies

TXN_CONCURRENCY

2.1 ARIES Write-Ahead Logging & Crash Recovery Protocol

Transactional storage engines decouple durability from random data-file I/O via the **ARIES protocol** (Algorithms for Recovery and Isolation Exploiting Semantics). Every modification generates a monotonically increasing Log Sequence Number (LSN), and every 8 KB/16 KB buffer pool page header records the $pageLSN$ of the most recent operation that mutated it:

- **The Write-Ahead Invariant:** A dirty buffer pool page cannot be flushed to non-volatile disk until the WAL record with $LSN \geq pageLSN$ has been persisted via `fsync()` ($pageLSN \leq flushedLSN$).
- **Three-Pass Crash Recovery:** (1) *Analysis Pass:* Scans forward from the last master checkpoint to reconstruct the Dirty Page Table (DPT) and active loser transactions. (2) *Redo Pass (Repeating History):* Scans forward from the smallest $recLSN$ in the DPT, replaying all logged changes—including uncommitted transactions—to return the database to its exact pre-crash state. (3) *Undo Pass:* Scans backward, reversing loser transactions in reverse chronological order and emitting Compensation Log Records (CLRs) so repeated crashes during recovery remain idempotent.

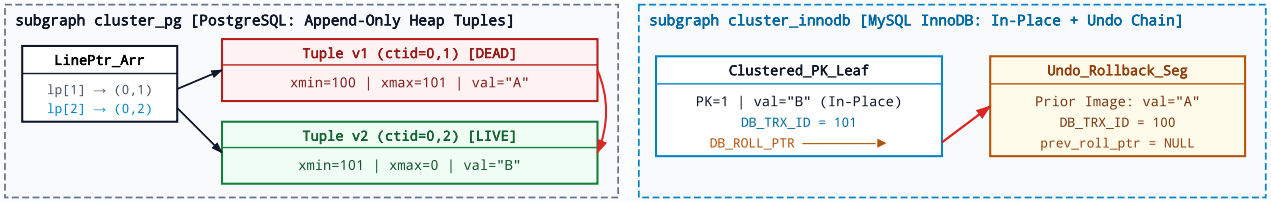


FIG 2.1: Multi-Version Concurrency Control (MVCC) Storage Topologies

```
layout=dot;
node[shape=record]
```

2.2 MVCC Storage Architectures: PostgreSQL vs. MySQL InnoDB

Dimension	PostgreSQL (Append-Only Tuple Versioning)	MySQL InnoDB (In-Place + Rollback Segment)
Update Mechanics	Writes a brand-new tuple version into the heap page with `xmin = current_txid`; sets the old tuple's `xmax = current_txid`.	Overwrites the clustered B+ Tree leaf record in place; serializes the before-image into an **Undo Log** linked via `DB_ROLL_PTR`.
Secondary Indexes	Point directly to physical heap `ctid`. Non-HOT updates require modifying every secondary index on the table.	Store the Primary Key value (logical pointer). Updates to non-indexed columns never touch secondary indexes.
Garbage Collection	Requires background `AUTOVACUUM` to reclaim dead tuples and freeze 32-bit `txid`s to prevent wraparound failure at 2^{31} txns.	Background purge threads clean undo segments once no active read view requires versions older than the undo record.
Failure Mode	Table and index bloat under high-churn updates; query latency degrades if long transactions hold back the `xmin` horizon.	Historical snapshot reads pay $O(k)$ pointer-chasing latency reconstructing rows across deep undo chains.

CONCURRENCY ANOMALY DEEP-DIVE: WRITE SKEW (G-SKEW) UNDER SNAPSHOT ISOLATION

PostgreSQL `REPEATABLE READ` implements **Snapshot Isolation (SI)**, which prevents Dirty Reads, Non-Repeatable Reads, and Lost Updates, but permits **Write Skew**. Consider an invariant requiring $\sum(on_call) \geq 1$ across doctors. Concurrent transactions T_1 and T_2 both read `COUNT(*) = 2` from their snapshot. T_1 updates `Alice.on_call = false`; T_2 updates `Bob.on_call = false`. Because T_1 and T_2 mutate disjoint rows, neither triggers a row-level write lock conflict. Both commit, violating the global invariant (`COUNT = 0`). PostgreSQL `SERIALIZABLE` prevents this via **Serializable Snapshot Isolation (SSI)**, placing non-blocking `SIREAD` predicate locks on index pages to detect dangerous $T_1 \rightarrow_{rw} T_2 \rightarrow_{rw} T_1$ dependency cycles and aborting one transaction with `SQLSTATE 40001`.

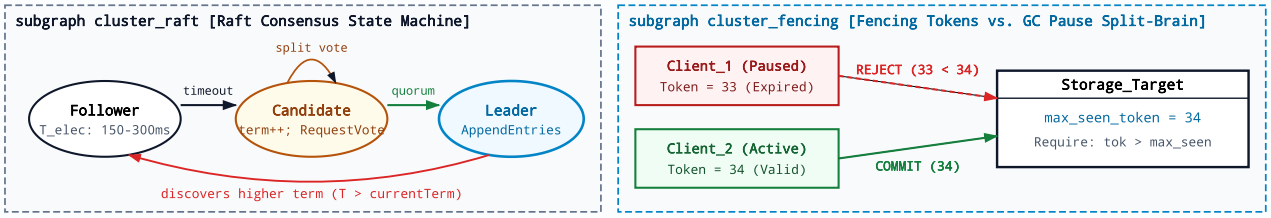


FIG 3.1: Raft Leader Election Automaton & Monotonic Fencing Token Enforcement `layout=dot; rankdir=LR`

3.1 Raft State Machine Mechanics (etcd, CockroachDB, TiKV, Kafka KRaft)

- **Leader Election & Log Completeness Guard:** Time is divided into strictly monotonic *Terms* (`currentTerm`). When a Follower's randomized election timer ($150–300\ ms$) expires without an `AppendEntries` heartbeat, it transitions to *Candidate*, increments `currentTerm`, votes for itself, and broadcasts `RequestVote` RPCs. A peer grants its vote **if and only if** $term_{\text{candidate}} \geq term_{\text{peer}}$ AND the candidate's log is at least as up-to-date as the peer's own log: $(lastLogTerm_{\text{cand}} > lastLogTerm_{\text{peer}}) \lor (lastLogTerm_{\text{cand}} = lastLogTerm_{\text{peer}} \land lastLogIndex_{\text{cand}} \geq lastLogIndex_{\text{peer}})$.
- **Log Replication & Divergence Repair:** The Leader accepts client writes, appends them to its local WAL, and issues `AppendEntries(term, prevLogIndex, prevLogTerm, entries[], leaderCommit)`. If a Follower's log does not match `(prevLogIndex, prevLogTerm)`, it rejects the RPC; the Leader decrements `nextIndex[follower]` (or jumps back by term) until finding the highest common ancestor index, overwriting divergent uncommitted follower entries.
- **The Section 5.4.2 Commit Safety Rule:** Once an entry from the *current term* is replicated on a majority quorum ($\lfloor N/2 \rfloor + 1$), it is committed. Crucially, a Raft Leader **never commits log entries from previous terms by counting replicas alone**; it only commits prior-term entries indirectly by committing at least one new entry from its current term (often an initial blank `no-op` entry emitted immediately upon winning election). Failing to enforce this allows a stale leader to overwrite a majority-replicated entry from an earlier term.

3.2 Crash Fault Tolerance (CFT) vs. Byzantine Fault Tolerance (PBFT)

- **CFT (Raft, Multi-Paxos, Zab):** Assumes a non-malicious fail-stop/fail-recovery model where nodes crash or networks drop/delay packets, but nodes never fabricate forged state. Requires $2f + 1$ nodes to tolerate f simultaneous failures (e.g., 5 nodes survive 2 crashes) because any two majority quorums of size $f + 1$ overlap in at least 1 node.
- **BFT (PBFT, HotStuff, Tendermint):** Tolerates arbitrary/adversarial nodes that equivocate (sending conflicting votes to different peers). Because f nodes may be unresponsive while another f nodes actively lie, a quorum of $N - f$ responses still contains f Byzantine votes. For honest responses to strictly outnumber forged ones ($N - 2f > f$), BFT requires $N \geq 3f + 1$ nodes and three communication phases (*Pre-Prepare*, *Prepare*, *Commit*) with $2f + 1$ cryptographic quorum certificates.

> **WHY TIME-BASED DISTRIBUTED LOCKS (REDLOCK) FAIL CORRECTNESS & HOW FENCING TOKENS FIX IT**
>
> Acquiring a lock in Redis with a TTL (`SET lock_key val NX PX 30000`) provides *liveness*, not mutual exclusion *safety*. If Client 1 acquires the lock and hits a 35-second JVM Stop-The-World Garbage Collection pause (or kernel scheduling delay), the lock TTL expires. Client 2 acquires the lock and writes to storage. When Client 1 wakes up, it believes it still holds the lock and overwrites Client 2's data. Correctness requires **Fencing Tokens**: the lock coordinator (ZooKeeper `zxid` or etcd `mod_revision`) returns a strictly increasing integer with every lock grant. The downstream storage resource rejects any write carrying a token $\leq max_committed_token$.

4. Message Queuing, Distributed Logs & Zero-Copy I/O

STREAMING_EOS

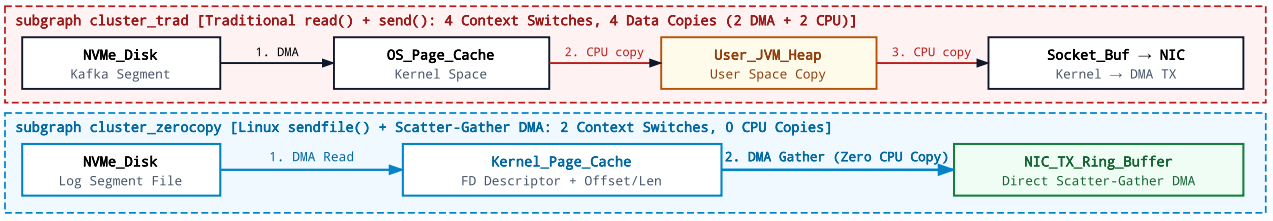


FIG 4.1: Kernel Data Path Comparison — Standard Socket I/O vs. sendfile() Zero-Copy DMA

```
layout=dot; rankdir=LR
```

4.1 AMQP Message Brokers (RabbitMQ) vs. Partitioned Commit Logs (Apache Kafka)

- **RabbitMQ (Per-Message Routing & Destructive Read):** Built on the Erlang BEAM VM using an actor-per-queue architecture. Exchanges route messages via bindings (direct, topic, fanout) into queues. Consumption is **destructive**: once a consumer emits `basic.ack`, the broker deletes the message from its storage index. Broker memory pressure triggers paging to disk, turning sequential workloads into random I/O spikes.
- **Apache Kafka (Immutable Partitioned Log & Offset Tracking):** Topics are sharded into append-only partitions stored as sequential 1 GB segment files (`.log`, `.index`, `.timeindex`). Consumption is **non-destructive**: consumers advance a 64-bit integer `offset`. Multiple independent consumer groups read the exact same partition concurrently from the OS Page Cache without lock contention.
- **High-Watermark & ISR Replication:** Each partition has 1 Leader and k Followers. Followers that stay caught up within `replica.lag.time.max.ms` belong to the **In-Sync Replicas (ISR)** set. Configuring `acks=all` and `min.insync.replicas=2` (with Replication Factor 3) guarantees a write is acknowledged only after at least 2 ISR brokers persist it. Consumers only read up to the *High Watermark* (the highest offset replicated across all ISR members).

4.2 End-to-End Exactly-Once Semantics (EOS) & Transactional Outbox

- **Kafka Idempotent Producer & Transactions:** Enabling `enable.idempotence=true` assigns each producer a unique `PID` (Producer ID) and attaches a monotonically increasing sequence number to every batch per partition. The broker deduplicates network retries in memory. Across partitions, a Transaction Coordinator uses a two-phase commit protocol (`__transaction_state` internal topic) to write output messages and commit consumer offsets atomically; consumers set `isolation.level=read_committed` to filter aborted transactions.
- **Solving the Dual-Write Bug (Transactional Outbox + CDC):** Updating PostgreSQL and publishing to Kafka in application code is unsafe—a crash between the two leaves systems inconsistent. The **Transactional Outbox Pattern** writes both the business entity mutation and the serialized event record into the same relational database within a single local ACID transaction:

```
    -- Atomic Local Transaction (PostgreSQL) + Debezium WAL Logical Decoding Relay BEGIN
TRANSACTION ISOLATION LEVEL READ COMMITTED; INSERT INTO ledger_transfers (tx_id, from_acct,
to_acct, amount_cents) VALUES ($1, $2, $3, $4) ON CONFLICT (tx_id) DO NOTHING; -- Idempotency
guard on client retry key INSERT INTO event_outbox (event_id, aggregate_id, topic,
payload_json, created_at) VALUES (gen_random_uuid(), $1, 'payments.transfers.v1', $5, NOW());
COMMIT;
```

PRIMARY SOURCES: Kreps, Narkhede, Rao ("Kafka: a Distributed Messaging System for Log Processing", NetDB '11) • Stripe Engineering ("Designing robust and predictable APIs with idempotency") • Debezium Outbox Architecture.

5. Linux Kernel Networking: epoll, Edge-Triggering & io_uring

KERNEL_NET_IO

5.1 From O(N) Poll Scans to O(1) epoll & Lock-Free io_uring

- **Why `select()` and `poll()` Collapse at C10K:** Every invocation copies an entire array of N file descriptors from user space into kernel space, scans all N socket wait queues linearly ($O(N)$), and copies the bitmask back. At $N = 100{,}000$ concurrent sockets, CPU cycles are exhausted by linear kernel scans.
- **`epoll` Internal Kernel Structures:** `epoll_create1()` allocates an in-kernel `eventpoll` struct containing two structures: (1) a **Red-Black Tree** (`rbr`) storing registered file descriptors via `epoll_ctl()` in $O(\log N)$ time, and (2) a doubly linked **Ready List** (`rdllist`). When a NIC hardware interrupt delivers a TCP segment, the kernel network stack invokes the registered callback `ep_poll_callback`, appending the active descriptor directly to `rdllist`. `epoll_wait()` simply dequeues ready items in $O(k)$ time ($k = active\ events$).
- **Edge-Triggered (`EPOLLET`) vs. Level-Triggered:** Level-triggered mode fires continuously while unread bytes remain in the socket buffer. Edge-triggered mode (used by Nginx and Envoy) notifies user space **only once per state transition** (e.g., socket buffer transitions from empty to non-empty). Workers must loop non-blocking `read()` calls until receiving `EAGAIN`, eliminating redundant `epoll_wait` wakeups.
- **`io_uring` Shared Ring Buffers:** Eliminates system-call transitions altogether by mapping two lock-free ring buffers between user and kernel memory: the **Submission Queue (SQ)** and **Completion Queue (CQ)**. With `IORING_SETUP_SQPOLL`, a kernel thread polls the SQ ring and dispatches NVMe and network I/O with zero syscalls.

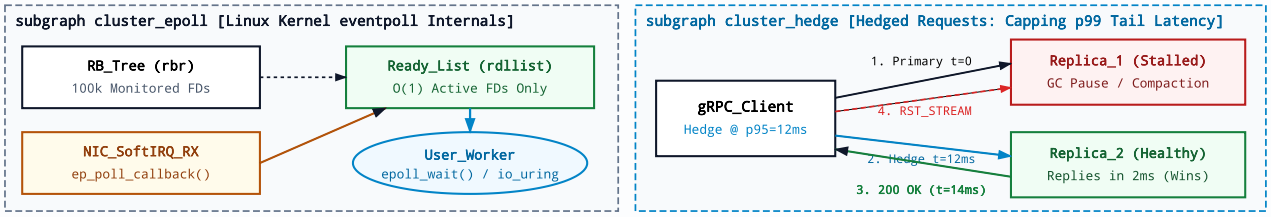


FIG 5.1: Kernel epoll Event Delivery & Speculative Hedged Request Cancellation

layout=dot; rankdir=LR

6. Performance Queueing Math, CoDel & Tail Latency

QUEUEING_MATH

6.1 Kingman's Formula, CoDel Bufferbloat Control & Fan-Out Amplification

- **Kingman's G/G/1 Queueing Equation:** In real services where neither request arrivals nor service durations follow smooth Poisson distributions, mean queue wait time (W_q) obeys Kingman's approximation: $W_q \approx (\rho / (1 - \rho)) \times ((C_a^2 + C_s^2) / 2) \times E[S]$, where $\rho = \lambda / \mu$ is server utilization, C_a and C_s are the coefficients of variation (standard deviation divided by mean) of arrival and service times, and $E[S]$ is mean service time. As $\rho \to 1.0$, the $\rho/(1-\rho)$ factor explodes nonlinearly, and bursty traffic ($C_a^2 > 1$) or GC jitter ($C_s^2 > 1$) multiplies queueing delay directly.
- **CoDel (Controlled Delay) & LIFO Load Shedding:** Deep unbounded queues cause *Bufferbloat*—workers waste CPU processing requests whose upstream client deadlines have already expired. Production RPC frameworks bound queues by **sojourn time** (monitoring minimum queue wait over a 100 ms sliding window), switching to LIFO dequeue during spikes and shedding stale requests immediately with `HTTP 503 / gRPC RESOURCE_EXHAUSTED`.
- **Tail Amplification & Hedged Requests:** If a root gateway fans out in parallel to $N = 100$ leaf microservices where each leaf has a 1% chance of a p_{99} stall, the probability that the root call experiences at least one slow leaf is $P(slow) = 1 - (1 - 0.01)^{100} = 63.4\%$. Issuing a **Hedged Request** to a second replica only after waiting for the p_{95} latency threshold cuts $p_{99.9}$ latency dramatically while increasing backend load by at most 5%.

Utilization ( )	Queue Multiplier ( )	Total Latency ( )	Bursty Traffic ( )
50% (Optimal	1.0x	4.0 ms (2ms service + 2ms	8.0 ms (2ms service + 6ms

7. Production Blueprint: Real-Time Event & Query Platform

REFERENCE_ARCH

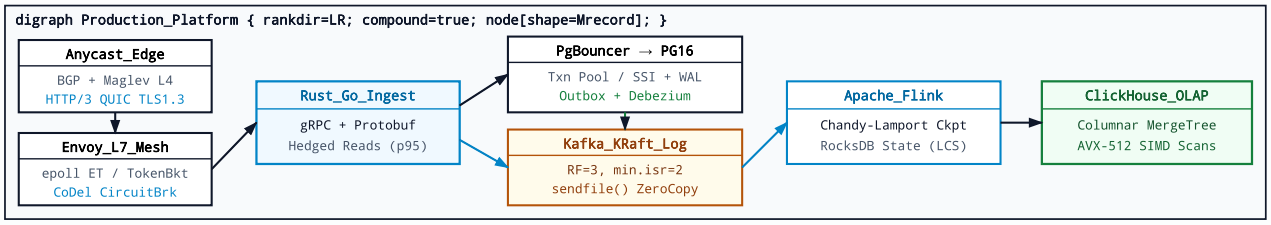


FIG 7.1: End-to-End Production Event Ingestion, Transactional CDC & OLAP Serving Pipeline `layout=dot; rankdir=LR; splines=ortho`

7.1 Execution Path & Mechanical Sympathy Across Tiers

1. **Anycast Edge & L4 Maglev Load Balancing:** Client connections route via BGP Anycast to the nearest PoP, where L4 load balancers use Google **Maglev consistent hashing** over 5-tuples to distribute packets across Envoy proxies without shared connection-tracking state, terminating TLS 1.3 (1-RTT handshake) and HTTP/3 over QUIC (eliminating TCP head-of-line blocking).
2. **L7 Envoy Proxy & Stateless API Fleet:** Envoy runs a shared-nothing *thread-per-core* event loop backed by `epoll` (`EPOLLET`) / `io_uring` , enforcing distributed token-bucket rate limits and CoDel adaptive concurrency limits before dispatching gRPC calls to Rust/Go services.
3. **Transactional OLTP & CDC Path:** Mutations route through **PgBouncer** in `transaction` pooling mode (multiplexing 10,000 app sockets onto ~50 PostgreSQL backend processes). PostgreSQL commits both the domain state and an `event_outbox` row in a single WAL flush. **Debezium** tails the PostgreSQL replication slot via `pgoutput` logical decoding and streams committed events into Kafka.
4. **Stateful Stream Processing & Columnar OLAP: Apache Flink** consumes Kafka partitions using asynchronous **Chandy-Lamport checkpoint barriers** for distributed exactly-once state snapshots stored in local **RocksDB** instances (configured with Leveled Compaction). Aggregated micro-batches flush into **ClickHouse** `MergeTree` columnar tables, executing vectorized SIMD scans over compressed column granules.

7.2 Authoritative Open-Source Codebases, Papers & Engineering Blogs

Architectural Domain	Reference Open-Source Codebases	Foundational Papers & Engineering Blogs
Storage Engines (B+ / LSM)	`facebook/rocksdb` , `scylladb/scylladb` , `postgres/postgres` (`src/backend/access/nbtree`)	O'Neil et al. ("The LSM-Tree", 1996); Alex Petrov (*Database Internals*); Meta RocksDB Blog.
WAL, MVCC & Isolation	PostgreSQL SSI (`src/backend/storage/lmgr/predicate.c`), MySQL InnoDB (`storage/innobase/trx`)	Mohan et al. ("ARIES", ACM TODS 1992); Cahill et al. ("Serializable Isolation for Snapshot Databases").
Consensus & Coordination	`etcd-io/raft` , `tikv/raft-rs` , `cockroachdb/cockroach` (`pkg/kv/kvserver`)	Ongaro & Ousterhout ("Raft", USENIX ATC '14); Martin Kleppmann ("How to do distributed locking").
Streaming & Zero-Copy	`apache/kafka` (KRaft + `FileRecords.java`), `redpanda-data/redpanda` (Seastar C++)	Kreps et al. ("Kafka", NetDB '11); Chandy & Lamport ("Distributed Snapshots", 1985); Stripe Idempotency Blog.
Kernel I/O & Tail Latency	`axboe/liburing` , `envoyproxy/envoy` , Linux Kernel (`fs/eventpoll.c`)	Dean & Barroso ("The Tail at Scale", CACM 2013); Nichols & Jacobson ("Controlling Queue Delay / CoDel").

RECOMMENDED CURRICULUM & RECORDED UNIVERSITY LECTURES

1. MIT 6.5840 (formerly 6.824) Distributed Systems — Prof. Robert Morris (YouTube lectures + Go labs implementing Raft, KV Service, and Sharded KV). **2. CMU 15-445 / 15-721 Database Systems** — Prof. Andy